

AI Chat Bot — Project Documentation

What is this project?

This is a simple rule-based AI Chat Bot implemented in C++. It simulates conversational behavior by matching cleaned user tokens (keywords) against predefined topics and returning a selected textual version from a JSON-like content file. It is not machine-learning based — instead it uses deterministic token processing, scoring, ranking and a small randomizer to vary responses.

Short flow summary

- Program starts at main()
- Menu management loads a JSON content file (line-by-line into a vector)
- User types a prompt
- The prompt is tokenized and cleaned
- Each JSON "Topic" heading is searched for token matches
- Matches get scored and topics are ranked
- The best matching topic's details are extracted (one of several "version_X" variants is chosen randomly)
- The chosen lines are printed as the bot's response

Why this project was created?

- To learn and demonstrate the building blocks of text-based chat interactions:
 - File I/O, tokenization, string cleaning, keyword matching, simple scoring, and menu control
- The project is intentionally simple enough to inspect every step and see how a rule-based chatbot can be constructed from basic functions.

Main goal

- To provide an educational, fun implementation that simulates an AI-like chat experience using only deterministic rules and content stored in a file.

Features

- Import content from a JSON-like file (Content.json)
- Tokenize and clean user input
- Remove common/stop words
- Find possible topics in file matching user tokens
- Score and rank topics by match percentage
- Extract content lines for the chosen topic
- Choose a random version (version_1..version_5) of the topic to reduce repetition
- Display the response to the user via a simple menu

Repository / Folder structure (as used in code)

- src/ (sources)
 - main.cpp
 - menus.cpp
 - menus_Managment.cpp
 - File_to_String.cpp
 - String_To_Tokens_Functions.cpp
 - Responce_Managment.cpp
- include/ (headers)
 - meuns_Managment.h
 - menus.h
 - File_to_String.h
 - String_To_Tokens_Functions.h
 - Responce_Managment.h
- data/Content.json

Modules (file-by-file explanation)

main.cpp

- Entry point of the program.
- Calls Menu_Managment() which runs the chat loop.
- Minimal content: `#include "../include/meuns_Managment.h" int main() { Menu_Managment(); }`

menus_Managment.cpp

- Acts as the coordinator for the application.
- Loads the data file into the global vector File using File_To_String(json_file_path).
- Runs the interactive loop:
 - On first pass shows a welcome screen (New_Chat_Menu())
 - Reads user prompt (Prompt_Input())
 - Gets bot response (Responce_Managment_System(prompt))
 - Displays output (Output_Display_Menu(response))
 - Declares two globals used by other modules:
 - vector File (holds lines read from Content.json)
 - string json_file_path (default: "data/Content.json")

menus.cpp

- Contains user interaction / display helpers:
 - Main_Menu(): prints the welcome header
 - Output_Display_Menu(vector): prints bot's multi-line response
 - Prompt_Input(): reads a full line from stdin (getline) and returns it

- New_Chat_Menu(): clears the console (system("cls")) and shows Main_Menu()

Note: system("cls") works on Windows. On Unix-like systems use "clear" or avoid console clearing.

File_to_String.cpp

- Reads the JSON file line-by-line into vector and returns it.
- File_To_String(string json_file_path):
 - Opens an ifstream
 - If it can't open, prints an error and returns an empty vector
 - Pushes each line into vector lines
 - Calls File_Display(lines) which prints all lines to stdout (for debugging)
 - Returns lines

Design note: This module treats Content.json as plain text (not parsed JSON). It relies on line structure to find "Topic" and "version_X" tokens.

String_To_Tokens_Functions.cpp

- Core text-processing utilities. Responsibilities:
 - Removing punctuation, quotes, commas, colons, braces, parentheses, slashes, etc.
 - Trimming leading/trailing whitespace
 - Lowercasing text
 - Splitting a string into word tokens
 - Removing common / stop words (examples: is, the, a, an, of, in, to, and, are, you, can, help, I, my, me)
 - Cleaning JSON value strings (Clean_Json_Details) to extract the actual content between quotes

Key functions:

- **Processed_String_To_Token(string str)**
 - Lowercases, tokenizes, cleans individual tokens and removes common words
 - Returns vector tokens suitable for matching
- **String_To_Token, Count_Number_of_Words, Clean_Array_of_Strings etc.**
- **Clean_Json_Details(string str)**
 - Designed for the specific Content.json formatting:
 - It takes line like: { "version_1": "Some text" }, and strips out the initial part and trailing characters to leave just the description.

Response_Managment.cpp

- The "reasoning" module that chooses what response to return based on a prompt.
- Main entry: Response_Managment_System(string prompt)
- Converts prompt into tokens using Processed_String_To_Token()

- Calls `Possible_Topic_In_Json(tokens)` to find candidate topic lines and scores
- If no topics, returns a default apology response
- Otherwise arranges candidate topics in descending order (`Arrange_Possible_Topics_Decending`)
- Extracts the matching topic's details (`Extract_Topic_Details_From_Json`) and returns them

Important helper functions:

- **`Possible_Topic_In_Json(vector tokens)`**
 - Iterates through every line in File
 - For each line, if the line contains "Topic" and contains any token, it increments points
 - Computes $\text{Points_Calculation} = (\text{points} / \text{\#tokens}) * 100$ and if points > 0, pushes {line_number, percent}
 - Returns vector<vector> where each inner vector is {line_number, percent}
- **`Arrange_Possible_Topics_Decending(...)`**
 - Simple bubble-style comparison that sorts the `Possible_Topics` array by the percent score in descending order
- **`Extract_Topic_Details_From_Json(...)`**
 - Picks the top matching topic (`Possible_Topics[0]`)
 - Adds 1 to the topic line index to start reading its "Detail" lines
 - Uses a randomizer (`rand()%5`) to choose which version set (`version_1..version_5`) to collect
 - Collects lines that match that chosen version and uses `Clean_Json_Details` to extract the text
 - Stops collecting once it hits the next "Topic" or the version lines end

Content.json

- Holds an array of objects. Each object contains:
 - "Topic": ""
 - "Detail": [{ "version_1": "...", }, { "version_2": "...", }, ...]
- Each topic is written across many lines; code expects a consistent form so `Clean_Json_Details` works
- Because file is parsed as lines rather than using a JSON parser, formatting (spaces, commas, quotes and line breaks) is significant

Example: How a prompt is handled (short)

- User: "Tell me about Pakistan movement"
- `Prompt_Input` returns the full string
- `Processed_String_To-Token`:
- Lowercases: "tell me about pakistan movement"
- Tokenizes to ["tell", "me", "about", "pakistan", "movement"]

- Cleans tokens, removes stop words (me, about) → tokens may become ["tell","pakistan","movement"]
- Possible_Topic_In_Json checks each line containing "Topic" if it contains any of these tokens
- The "Topic" line " "Topic": "pakistan movement"," will match both "pakistan" and "movement" and get a high percent
- Topic ranking chooses the best matching topic (pakistan movement)
- Extract_Topic_Details_From_Json chooses one "version_X" set at random and returns those lines (Clean_Json_Details used to strip the JSON noise)
- Output_Display_Menu prints the selected lines

```

=====
===== Hi, I am Ai Chat Bot =====
=====
----- Ai Chat Bot -----

How can I help you today?

----- User -----

User: Tell me about Pakistan movement
----- Ai Chat Bot -----

The Political Evolution of the Pakistan Movement
Buddy, the Pakistan Movement evolved through decades of political awa
kening.
When British rule began weakening and democratic reforms were introdu
ced, it became clear that the Muslim minority risked being overshadow
ed in a Hindu majority democracy.
The All India Muslim League emerged as a platform to voice Muslim con
cerns, arguing that political safeguards weren't enough t true se
curity required an independent state.
The Lahore Resolution of 1940 became the turning point, declaring tha
t Muslims were a distinct nation deserving a homeland.
From that point on, the movement intensified, driven by strategic pol
itical negotiations, public mobilization, and an unwavering demand fo
r autonomy.
It wasn't a sudden revolt; it was a carefully built political reali
ty that shaped the subcontinent.

----- User -----

User:

```